
The Transformer, Completed

Decoder · Positional Encoding · Training

Week 2 study notes, LLM course, IITM BS in Data Science

Based on the lectures of **Prof. Mitesh M. Khapra**
AI4Bharat, Department of CSE, IIT Madras

(lecture slides: Modules 16.2 to 16.4, mapped here to Modules 2.1 to 2.3)

How to read these notes

- **Blue boxes:** definitions and key equations (exam material).
- **Green boxes:** intuition, i.e. *why* the idea exists.
- **Grey boxes:** step-by-step derivations.
- **Red boxes:** common mistakes and exam traps.
- **Orange boxes:** material *beyond* the slides (modern practice, systems view). Useful context; not examinable from these slides.
- Shapes are written in grey brackets, e.g. $[T \times 512]$. The notation page (p. 3) fixes the notation once.

Contents

Before We Start: Notation and the Running Example	3
2.1 The Decoder	4
2.1.1 Where the Decoder Fits	4
2.1.2 The Decoder Stack: Three Sublayers per Layer	4
2.1.3 Teacher Forcing	5
2.1.4 Masked Multi-Head Self-Attention	6
2.1.5 Multi-Head Cross-Attention	9
2.1.6 Feed-Forward Network	10
2.1.7 Number of Parameters per Decoder Layer	11
2.1.8 Decoder Output: Linear Layer + Softmax	11
2.1.9 Training Objective and Inference Procedure	12
2.2 Positional Encoding	13
2.2.1 Why Positional Information Is Missing	13
2.2.2 How Should We Fill p_j ? Three Naive Ideas	13
2.2.3 The Sinusoidal Encoding Function	14
2.2.4 The Heat Map and the Special Case $j = 0$	15
2.2.5 Distance Properties	16
2.2.6 Relative Position as a Linear Map	17
2.3 Training the Transformer	19
2.3.1 How Deep Is the Transformer?	19
2.3.2 Batch Normalization: Recap	19
2.3.3 Limitations of Batch Normalization	20
2.3.4 Layer Normalization	20
2.3.5 Residual Connections	21
2.3.6 The Complete Layers	22
2.3.7 The Complete Architecture: Final Remarks	22
One-Page Revision Sheet	26
Practice	27
Part A: Numerical Problems	27
Part B: Subjective Questions	31
Part C: Multiple-Choice Questions	33
Part C: Answer Key	35

List of Figures

2.1.1 The big picture	4
2.1.2 Decoder stack (left) and the three sublayers inside each decoder layer (right) . . .	5
2.1.3 The slides' example	6
2.1.4 Masked scaled dot-product attention, as in the slides	7
2.1.5 The matrix $A + M$ for the running example (scaling omitted for readability) . . .	8
2.1.6 Cross-attention	10
2.1.7 One decoder layer with its parameter budget, as on the slides	11
2.2.1 Without positional information, shuffling the input just shuffles the output	13
2.2.2 Four sine channels of the positional encoding	15
2.2.3 $PE_{(j,i)}$ for $j = 0..8$ (rows) and entries $i = 0..63$ (columns)	15

2.2.4	The full picture: all 512 entries (vertical) for positions 0 to 99 (horizontal)	16
2.2.5	Left: actual Euclidean distances $\ p_j - p_k\ $ for the 5-word sentence	17
2.3.1	Counting the transformer as a plain deep network, as on the slides: 42 layers . . .	19
2.3.2	The (l, i, j) cube from the slides	20
2.3.3	The Add & Norm wrapper used around every sublayer	22
2.3.4	The complete transformer (“Attention Is All You Need”), redrawn from the slides	23
2.3.5	Where the parameters live	24

Before We Start: Notation and the Running Example

Week 2 finishes the transformer that Week 1 started. Week 1 built the **encoder** (self-attention, multi-head attention, the feed-forward network). This week covers the three remaining pieces:

1. the **decoder**: how the model *generates* the output sentence (Module 2.1);
2. **positional encoding**: how the model learns word *order* (Module 2.2);
3. **training tricks**: residual connections and layer normalization, which make a 42-layer network trainable (Module 2.3).

The running example

The lectures translate English to Tamil (transliterated):

Source (encoder input): I enjoyed the movie transformers

Target (decoder output): Naan transfarmar padaththai rasiththen

The decoder input is the target *shifted right*, starting with a special start token <Go>: <Go> Naan transfarmar padaththai rasiththen.

Hyper-parameters of the base transformer

Symbol	Meaning	Value
d_{model}	width of every token vector in the model	512
h	number of attention heads	8
$d_k = d_v$	per-head query/key and value width, d_{model}/h	64
d_{ff}	hidden width of the feed-forward network	2048
N	layers in the encoder stack, and again in the decoder stack	6
$ V $	target vocabulary size (slides' example)	37,000
T	sequence length (number of tokens)	varies

Matrix convention (important)

The slides use the **column convention**: each token is a *column* vector.

$$H = \begin{bmatrix} h_1 & h_2 & \cdots & h_T \end{bmatrix} \in \mathbb{R}^{d_{\text{model}} \times T}, \quad Q = W_Q H \in \mathbb{R}^{d_k \times T}, \quad W_Q \in \mathbb{R}^{d_k \times d_{\text{model}}}.$$

Under this convention the attention scores are $A = Q^T K \in \mathbb{R}^{T \times T}$ and the output is

$$Z = \text{softmax}(A) V^T \in \mathbb{R}^{T \times d_v}.$$

That is why the slides write V^T at the end.

Watch out: Two conventions, same maths

PyTorch and most papers use the **row convention**: tokens are *rows*, $H \in \mathbb{R}^{T \times d_{\text{model}}}$, $Q = HW_Q$, and $Z = \text{softmax}(QK^T/\sqrt{d_k})V$. Both give the same $T \times d_v$ result. In an exam, stick to the convention the question uses, and check that the shapes multiply. These notes follow the slides (columns) unless a box says otherwise.

Module 2.1

The Decoder

(slides p. 1–11, lecture Module 16.2)

2.1.1 Where the Decoder Fits

The encoder reads the whole source sentence and outputs one contextual vector per source word, $E = [e_1, \dots, e_{T_s}]$. The **decoder** then writes the target sentence *one token at a time*. At every step it looks at two things:

1. what it has written so far (the target prefix), and
2. the encoder's representation E of the source.

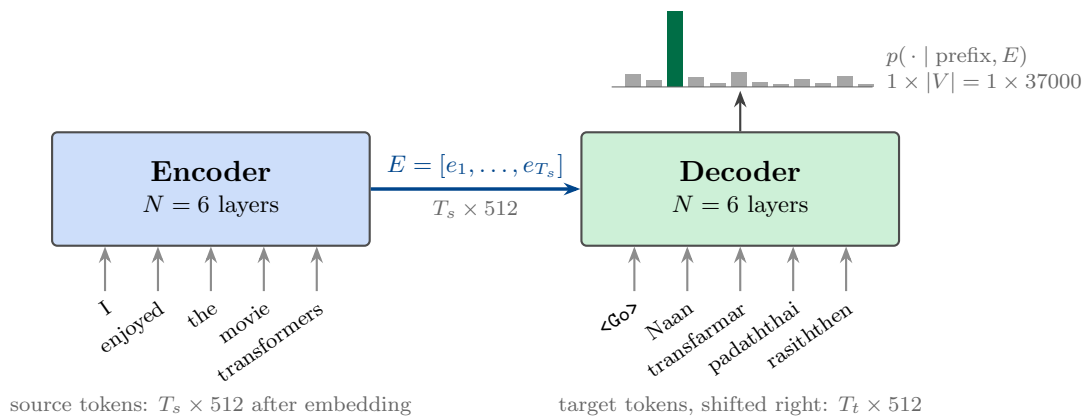


Figure 2.1.1: The big picture. The encoder turns the source into E once. The decoder uses E and the target prefix to output a probability distribution over the *whole* target vocabulary at each position.

What is the output dimension of the decoder?

At each position the decoder outputs one probability for every word in the target vocabulary:

$$1 \times |V| \quad \text{e.g. } 1 \times 37000.$$

For all T_t target positions together this is a $T_t \times |V|$ matrix; every row sums to 1.

Intuition: Why a separate decoder at all?

Input and output sentences differ in length, word order and vocabulary (English vs Tamil). The encoder's job is *understanding*: it builds a summary of the source that sees every word in both directions. The decoder's job is *writing*: it produces one word at a time, and may only use the words it has already written. These are two different jobs, so they get two different stacks.

2.1.2 The Decoder Stack: Three Sublayers per Layer

(slides p. 2)

The decoder is a stack of $N = 6$ identical layers. Each layer has **three sublayers** (an encoder layer has two):

1. **Masked multi-head self-attention**: each target word looks at the target words *up to and including itself* (Section 2.1.4).

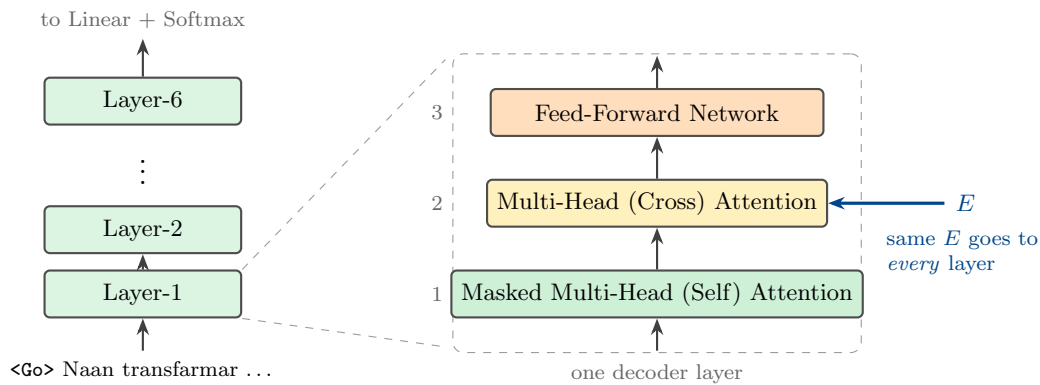


Figure 2.1.2: Decoder stack (left) and the three sublayers inside each decoder layer (right). The Add & Norm blocks around each sublayer are left out here; they are added in Module 2.3.

2. **Multi-head cross-attention:** each target word looks at *all* source words through E (Section 2.1.5).
3. **Feed-forward network:** the same 2-layer MLP applied to each position separately.

Decoder layer ℓ (without Add & Norm)

With $H^{(0)} = \text{target embeddings} + \text{positional encodings}$,

$$S^{(\ell)} = \text{MaskedMHA}(H^{(\ell-1)}) \quad \text{queries, keys, values all from the target} \quad (2.1.1)$$

$$C^{(\ell)} = \text{CrossMHA}(S^{(\ell)}, E) \quad \text{queries from } S^{(\ell)}, \text{ keys/values from } E \quad (2.1.2)$$

$$H^{(\ell)} = \text{FFN}(C^{(\ell)}) \quad \text{position-wise} \quad (2.1.3)$$

Every $H^{(\ell)}$ keeps the same shape, $[T_t \times 512]$. That is what makes the layers stackable.

2.1.3 Teacher Forcing

(slides p. 3)

Why is the target sentence an *input* to the decoder?

This is the question the slides open with. The decoder is trained to predict the target sentence, yet the target sentence is also fed *in*. Why?

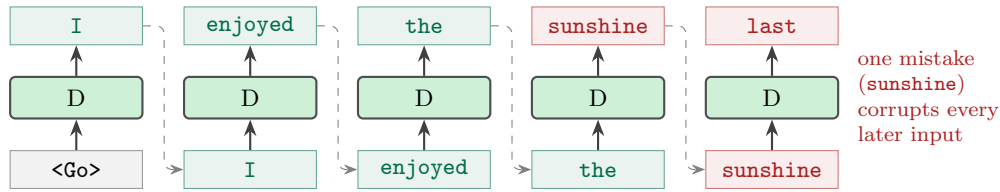
Option 1: free-running (autoregressive) training. Feed the decoder only its *own previous prediction*. The trouble is the chain rule of probability:

$$p(y_1, \dots, y_T | x) = \prod_{t=1}^T p(y_t | y_1, \dots, y_{t-1}, x).$$

Every factor is conditioned on the earlier outputs. If y_3 is wrong, then $y_4, y_5, \dots$ are all predicted from a wrong prefix, so errors **accumulate**. The model eventually learns to fix this, but training takes a very long time. Early in training almost every prefix is wrong, so the later positions receive useless learning signal.

Option 2: teacher forcing. During *training*, feed the decoder the **ground-truth** previous token, whatever it predicted. Position t gets the true y_{t-1}^* and is trained to predict y_t^* .

(a) Free-running: input = own previous prediction



(b) Teacher forcing: input = ground truth

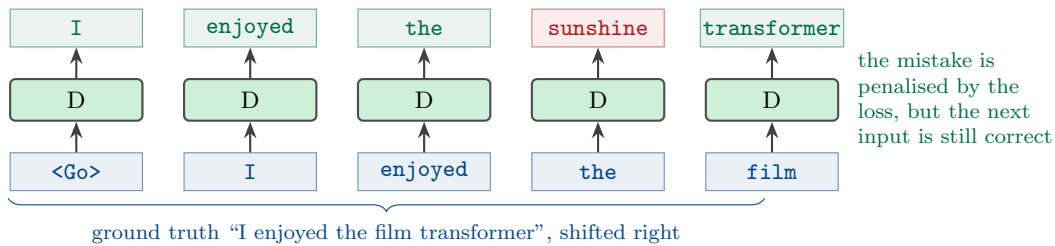


Figure 2.1.3: The slides' example. Target: *I enjoyed the film transformer*. (a) In free-running mode the wrong prediction **sunshine** becomes the next input, so the model continues with "...last night". (b) With teacher forcing each position is conditioned on the true prefix, so one mistake stays one mistake.

Intuition: Learning to drive with an instructor

Teacher forcing is like a driving instructor who corrects the steering after every mistake. You still get feedback (the loss) on each wrong turn, but you never drift into the wrong neighbourhood and waste the rest of the lesson there. Every position learns something useful from the first iteration on.

Watch out: Teacher forcing is training-only

At **inference** there is no ground truth to feed in. The decoder *must* run autoregressively on its own predictions, one token at a time (Section 2.1.9). This train/test mismatch is called **exposure bias**: the model never practised recovering from its own errors.

The hidden bonus: all positions in parallel

With teacher forcing, the *entire* decoder input is known before training starts. So we can feed all T_t positions at once and get all T_t predictions in a **single forward pass**, much like the encoder. There is one catch: position t must not be allowed to peek at the input tokens after it, because those include the answer it is supposed to predict. The next section fixes exactly this with a **mask**.

2.1.4 Masked Multi-Head Self-Attention

(slides p. 4–7)

Recap: what is the same as the encoder

As in the encoder, the target embeddings $h_1, \dots, h_T$ (now the *target* words) are multiplied by learned matrices to get queries, keys and values. For head 1 of the first sublayer (subscript 1 = "first attention sublayer"):

$$Q_1 = W_{Q_1} H \quad [d_k \times T], \quad K_1 = W_{K_1} H \quad [d_k \times T], \quad V_1 = W_{V_1} H \quad [d_v \times T], \quad (2.1.4)$$

with $H \in \mathbb{R}^{512 \times T}$ and $W_{Q_1}, W_{K_1} \in \mathbb{R}^{64 \times 512}$, $W_{V_1} \in \mathbb{R}^{64 \times 512}$. The score matrix is

$$A = \frac{Q_1^T K_1}{\sqrt{d_k}} \quad [(T \times d_k)(d_k \times T) = T \times T], \quad A_{ij} = \frac{q_i \cdot k_j}{\sqrt{d_k}}. \quad (2.1.5)$$

Entry A_{ij} measures how much token i “wants” information from token j .

Derivation: Why divide by $\sqrt{d_k}$? (the “Scale” block on the slides)

Suppose the components of q and k are independent with mean 0 and variance 1. Then

$$\text{Var}(q \cdot k) = \text{Var}\left(\sum_{m=1}^{d_k} q_m k_m\right) = \sum_{m=1}^{d_k} \underbrace{\text{Var}(q_m k_m)}_{=1} = d_k.$$

With $d_k = 64$ the raw scores have standard deviation 8. Softmax of numbers that far apart is almost one-hot, so the gradients are close to zero. Dividing by $\sqrt{d_k} = 8$ brings the standard deviation back to 1 and keeps the softmax in its responsive range.

The one important difference: the mask

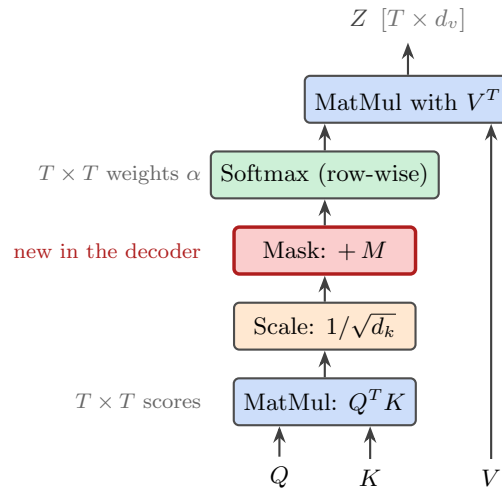


Figure 2.1.4: Masked scaled dot-product attention, as in the slides. The only change from the encoder is the red *Mask* block, placed after scaling and *before* the softmax.

Masked self-attention

$$Z = \text{softmax}(A + M) V_1^T \quad M_{ij} = \begin{cases} 0 & j \leq i \quad (\text{past or present: allowed}) \\ -\infty & j > i \quad (\text{future: blocked}) \end{cases}$$

Shapes: $A, M \in \mathbb{R}^{T \times T}$, $V_1^T \in \mathbb{R}^{T \times d_v}$, $Z \in \mathbb{R}^{T \times d_v}$. The goal: attention weight $\alpha_{ij} = 0$ for every value vector v_j with $j > i$.

Where should the mask go? (the slides’ question)

There are three candidate places: the input, the output, or somewhere in between.

- **At the input** (e.g. zeroing future embeddings): a zero vector still has a key $k_j = W_K \cdot 0 = 0$ with score $q_i \cdot 0 = 0$. That score gets weight $e^0 / \sum > 0$ after the softmax, so the position is not actually removed.

- **After the softmax** (setting $\alpha_{ij} = 0$): the row no longer sums to 1, so it would need re-normalising. That works, but it is an extra step.
- **Just before the softmax, as $-\infty$:** $e^{-\infty} = 0$, so the softmax itself gives exactly zero weight to future tokens *and* re-normalises over the allowed ones. One addition does everything. This is the choice in the slides.

Derivation: Why $-\infty$ gives exactly zero weight

For row i of $A + M$, the softmax is

$$\alpha_{ij} = \frac{e^{A_{ij}+M_{ij}}}{\sum_{m=1}^T e^{A_{im}+M_{im}}} = \frac{e^{A_{ij}} \mathbb{1}[j \leq i]}{\sum_{m \leq i} e^{A_{im}}},$$

since $e^{A_{im}-\infty} = 0$ for $m > i$. So the weights over positions $1..i$ still sum to 1, and positions $i+1..T$ get exactly 0. In code, $-\infty$ is a large negative number such as -10^9 , or `float('-inf')`.

		key j (column)				
		<Go>	Naan	transfarmar	padaththai	rasiththen
query i (row)	<Go>	$q_1 \cdot k_1$	$-\infty$	$-\infty$	$-\infty$	$-\infty$
	Naan	$q_2 \cdot k_1$	$q_2 \cdot k_2$	$-\infty$	$-\infty$	$-\infty$
	transfarmar	$q_3 \cdot k_1$	$q_3 \cdot k_2$	$q_3 \cdot k_3$	$-\infty$	$-\infty$
	padaththai	$q_4 \cdot k_1$	$q_4 \cdot k_2$	$q_4 \cdot k_3$	$q_4 \cdot k_4$	$-\infty$
	rasiththen	$q_5 \cdot k_1$	$q_5 \cdot k_2$	$q_5 \cdot k_3$	$q_5 \cdot k_4$	$q_5 \cdot k_5$

red: $j > i$, future, set to $-\infty$

green: $j \leq i$, past and present, kept

Figure 2.1.5: The matrix $A + M$ for the running example (scaling omitted for readability). Row i answers “which tokens may token i look at?”. <Go> sees only itself. **rasiththen** sees everything before it.

The mask matrix

$$M = \begin{bmatrix} 0 & -\infty & -\infty & \cdots & -\infty \\ 0 & 0 & -\infty & \cdots & -\infty \\ \vdots & & \ddots & \ddots & \vdots \\ 0 & 0 & \cdots & 0 & -\infty \\ 0 & 0 & \cdots & 0 & 0 \end{bmatrix} \in \mathbb{R}^{T \times T}$$

Zeros on and below the diagonal, $-\infty$ strictly above it. In the slides’ words, “a triangular matrix with one half all zeros and the other half all negative infinity”.

Intuition: Masking makes parallel training look sequential

The mask gives every row the view it would have *at inference time*. Row i sees tokens $1..i$, exactly what the model would have generated up to step i . So one parallel forward pass computes the same thing as T separate sequential steps. That is the whole reason transformers train so much faster than RNN decoders.

Worked mini-example

Take $T = 3$ and $d_k = 2$, and use the row convention for easy arithmetic (q_i, k_i, v_i are rows):

$$Q = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}, K = \begin{bmatrix} 1 & 0 \\ 1 & 1 \\ 0 & 1 \end{bmatrix}, V = \begin{bmatrix} 1 & 2 \\ 3 & 0 \\ 0 & 4 \end{bmatrix}.$$

Step 1: Scaled scores. $\frac{QK^T}{\sqrt{2}} = \begin{bmatrix} 0.707 & 0.707 & 0 \\ 0 & 0.707 & 0.707 \\ 0.707 & 1.414 & 0.707 \end{bmatrix}$

Step 2: Add the mask. $\begin{bmatrix} 0.707 & -\infty & -\infty \\ 0 & 0.707 & -\infty \\ 0.707 & 1.414 & 0.707 \end{bmatrix}$

Step 3: Row-wise softmax. Row 1: $[1, 0, 0]$. Row 2: $e^0 = 1$, $e^{0.707} = 2.028$, so $[0.330, 0.670, 0]$.
Row 3: $e^{0.707} = 2.028$, $e^{1.414} = 4.113$, sum = 8.169, so $[0.248, 0.504, 0.248]$.

Step 4: Weighted sum of values.

$$Z = \alpha V = \begin{bmatrix} 1 \cdot [1, 2] \\ 0.330[1, 2] + 0.670[3, 0] \\ 0.248[1, 2] + 0.504[3, 0] + 0.248[0, 4] \end{bmatrix} = \begin{bmatrix} 1.000 & 2.000 \\ 2.340 & 0.660 \\ 1.759 & 1.490 \end{bmatrix}.$$

Token 1's output is just v_1 , since it can only see itself. Token 2 mixes v_1 and v_2 and never touches v_3 .

Multi-head version

As in the encoder, this runs in $h = 8$ heads, each with its own $W_Q^{(r)}, W_K^{(r)}, W_V^{(r)}$ ($r = 1..8$) and the *same* mask M . The eight $T \times 64$ outputs are concatenated into $T \times 512$ and multiplied by an output matrix $W_{O_1} \in \mathbb{R}^{512 \times 512}$. The slides write the whole sublayer as $f(Q_1, K_1, V_1; \text{Mask}; W_{O_1})$.

Watch out: The encoder masks too, but for padding

(slides p. 5, footnote) In practice the encoder's attention also uses a mask, but for a different purpose. Sentences in a batch are padded to a common length T with **<pad>** tokens, and a **padding mask** puts $-\infty$ in the *columns* of pad tokens so nobody attends to them. The decoder's triangular mask is a **causal (look-ahead) mask**. The decoder uses both.

2.1.5 Multi-Head Cross-Attention

(slides p. 8)

After masked self-attention we have $S = [s_1, \dots, s_{T_t}]$: each target word, enriched with information from the target words before it. Now each target word needs to look at the *source* sentence. The inputs are:

- $s_1, \dots, s_{T_t}$ from the decoder's own masked self-attention sublayer;
- $e_1, \dots, e_{T_s}$ from the **top layer of the encoder**. The same E is shared with **all six** decoder layers.

Cross-attention (encoder–decoder attention)

$$Q_2 = W_{Q_2} S \quad [d_k \times T_t] \quad \text{queries come from the decoder} \quad (2.1.6)$$

$$K_2 = W_{K_2} E \quad [d_k \times T_s] \quad \text{keys come from the encoder} \quad (2.1.7)$$

$$V_2 = W_{V_2} E \quad [d_v \times T_s] \quad \text{values come from the encoder} \quad (2.1.8)$$

$$Z = \text{softmax}\left(\frac{Q_2^T K_2}{\sqrt{d_k}}\right) V_2^T \quad [(T_t \times T_s)(T_s \times d_v) = T_t \times d_v] \quad (2.1.9)$$

The heads are concatenated and projected with W_{O_2} , giving $T_t \times 512$ again. **No causal mask is needed:** the whole source sentence is known before decoding starts.

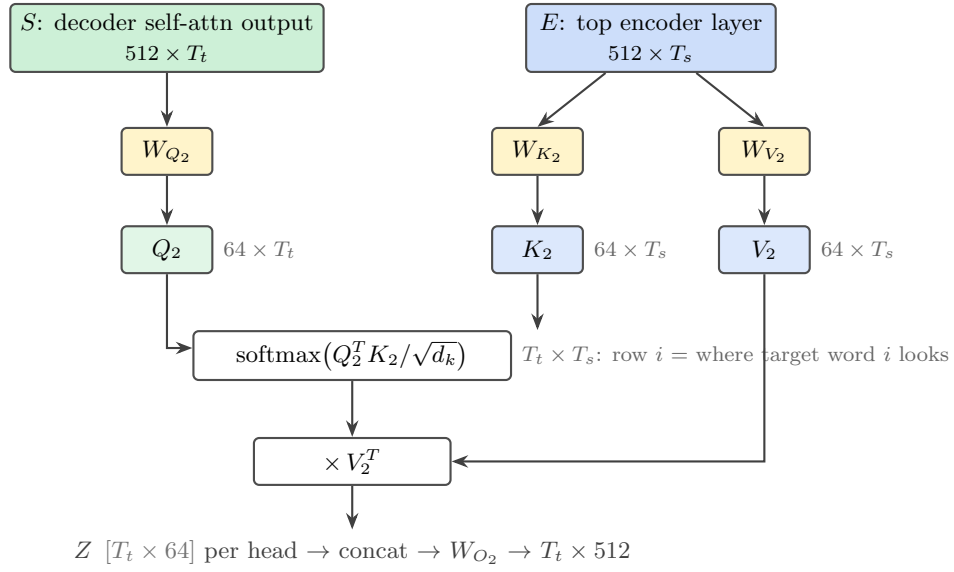


Figure 2.1.6: Cross-attention. The decoder provides the *questions* (Q_2). The encoder provides the *searchable index* (K_2) and the *content* (V_2). The weight matrix is rectangular, $T_t \times T_s$, because the two sentences can have different lengths.

Intuition: A library search

Think of the encoder output as a library. Each source word has a catalogue card (key) and the actual book (value). While writing *rasiththen* (“enjoyed”), the decoder issues a query that roughly says “I need the verb and its sense”. That query matches the key of *enjoyed* best, so mostly that word’s value is read. Queries must come from the side that is *asking* (the decoder, which is generating). Keys and values must come from the side being *searched* (the source).

Watch out: Cross-attention shapes

In cross-attention the weight matrix is $T_t \times T_s$, not square. If an answer shows a $T_s \times T_s$ or $T_t \times T_t$ matrix in cross-attention, the Q/K sources have been mixed up. The output always has T_t rows, one per *target* position, because the queries set the number of outputs.

2.1.6 Feed-Forward Network

Finally, each position’s vector goes through the same two-layer network, independently:

$$\text{FFN}(x) = \max(0, xW_1 + b_1) W_2 + b_2, \quad W_1 \in \mathbb{R}^{512 \times 2048}, W_2 \in \mathbb{R}^{2048 \times 512}. \quad (2.1.10)$$

It expands to 2048 dimensions, applies ReLU, and projects back to 512. “Position-wise” means the FFN never mixes tokens. All mixing across tokens happens in the attention sublayers. The output vectors O of the topmost layer feed the final linear layer.

2.1.7 Number of Parameters per Decoder Layer

(slides p. 10)

Derivation: Counting parameters in one decoder layer

Masked multi-head self-attention. Each head has $W_Q, W_K, W_V \in \mathbb{R}^{64 \times 512}$: $3 \times 64 \times 512 = 98,304$ per head. Eight heads: 786,432. The output projection $W_O \in \mathbb{R}^{512 \times 512}$ adds 262,144.

$$786,432 + 262,144 = 1,048,576 = 4 \times 512^2 \approx 1\text{M} \quad (+ 2048 \text{ biases} = 1,050,624).$$

Cross-attention. Same structure, another $\approx 1\text{M}$. (Its inputs differ; its matrix sizes do not.)

FFN. (slides p. 10)

$$\underbrace{512 \times 2048}_{W_1} + \underbrace{2048}_{b_1} + \underbrace{2048 \times 512}_{W_2} + \underbrace{512}_{b_2} = 2 \times (512 \times 2048) + 2048 + 512 = 2,099,712 \approx 2 \times 10^6.$$

Layer norms (Module 2.3): 3 per decoder layer $\times (\gamma, \beta \in \mathbb{R}^{512}) = 3,072$. Negligible.

Total $\approx 1\text{M} + 1\text{M} + 2\text{M} = 4\text{M}$ per decoder layer (exactly 4,204,032 with all biases).

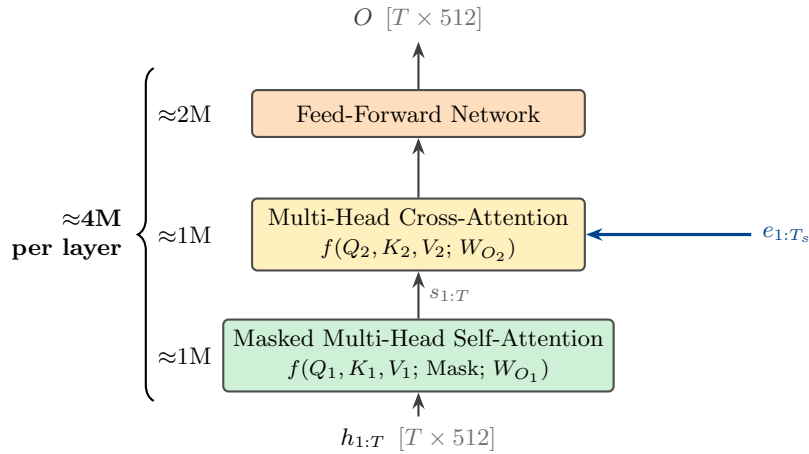


Figure 2.1.7: One decoder layer with its parameter budget, as on the slides.

2.1.8 Decoder Output: Linear Layer + Softmax

(slides p. 11)

The top decoder layer outputs $O \in \mathbb{R}^{T \times 512}$. Each row is turned into a distribution over words:

Output layer

$$\text{logits} = O W_D \quad [(T_t \times 512)(512 \times |V|) = T_t \times |V|], \quad P = \text{softmax}_{\text{row-wise}}(\text{logits}).$$

Row t of P is $p(y_t \mid y_{<t}, x)$: a $1 \times |V|$ vector.

Watch out: W_D is huge

$W_D \in \mathbb{R}^{512 \times |V|}$ has $512 \times 37,000 = 18,944,000 \approx \mathbf{19 \text{ million}}$ parameters. The slides point out this is about 19M of the model's $\approx 65\text{M}$ total, nearly 30% in a single matrix. The whole 6-layer decoder has only about 25M.

Beyond the slides: weight tying

The original paper *shares* one matrix between the source embedding, the target embedding and W_D (they use a joint source/target vocabulary). Word w 's embedding row and its output column are the same vector, which saves about 38M parameters. The “65M” figure assumes this sharing (Section 2.3.7).

2.1.9 Training Objective and Inference Procedure**Training: one forward pass, T_t losses**

With teacher forcing and the causal mask, a single pass produces $P \in \mathbb{R}^{T_t \times |V|}$. The loss is the cross-entropy summed over positions:

$$\mathcal{L} = - \sum_{t=1}^{T_t} \log P_{t, y_t^*} \quad (\text{the probability the model gave to the correct word at each position}).$$

Inference: one token at a time

At test time the target is unknown, so the decoder is run in a loop:

Step	Decoder input so far	Predicted next token	Action
1	<Go>	Naan	append
2	<Go> Naan	transfarmar	append
3	<Go> Naan transfarmar	padaththai	append
4	<Go> Naan transfarmar padaththai	rasiththen	append
5	<Go> ... rasiththen	<eos>	stop

The encoder runs **once**. The decoder runs once **per output token**, and at each step only the *last* row of P is used. The next token is chosen from that row by arg max (greedy) or by sampling.

Beyond the slides: why decoding is the expensive part

Since generation is sequential, producing T tokens takes T decoder passes. Implementations avoid recomputing old work in two ways. (1) The cross-attention K_2, V_2 depend only on E , so they are computed once per sentence and reused at every step. (2) The self-attention keys and values of past tokens never change (because of the causal mask), so they are stored in a **KV cache**, and each new step computes only the new token's q, k, v . The causal mask is what makes the KV cache valid. This is a core idea in LLM inference engineering.

Module 2.2

Positional Encoding

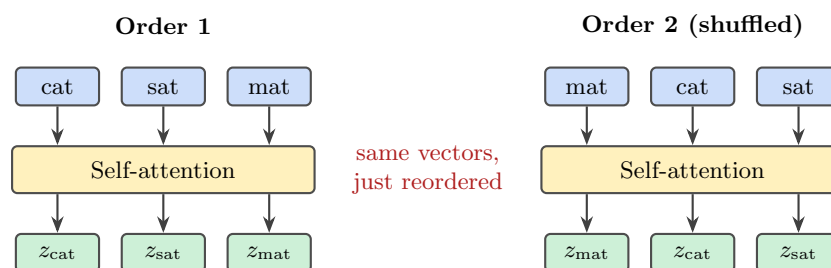
(slides p. 12–19, lecture Module 16.3)

2.2.1 Why Positional Information Is Missing

(slides p. 13)

In an RNN, word order is built into the computation. h_t is computed from h_{t-1} , so the hidden state “knows” it comes after everything before it. A transformer has no such recurrence:

- every token is processed *at the same time*;
- attention scores $q_i \cdot k_j$ depend only on the *contents* of tokens i and j , not on where they sit;
- so neither the encoder nor the decoder has any information about position.



fix: add a position vector p_j to each embedding h_j

Figure 2.2.1: Without positional information, shuffling the input just shuffles the output. Each word gets exactly the same vector z in both orders, so “cat sat mat” and “mat cat sat” look identical to the model.

Watch out: “Invariant” vs “equivariant”

The slides say self-attention is *permutation-invariant*. Strictly, the output *sequence* is permutation-**equivariant**: permute the inputs and the outputs are permuted the same way (z_{cat} stays z_{cat}). What is *invariant* is each token’s output vector, and anything order-free built on top, such as a sum. Either way the conclusion holds: the model cannot tell the orders apart.

The fix: add a position vector

For each position j , build a vector $p_j \in \mathbb{R}^{512}$ and **add** it to the word embedding:

$$h'_j = h_j + p_j \quad [512+512 \rightarrow 512] \quad (2.2.1)$$

Adding (not concatenating) keeps the width at 512, so nothing downstream changes.

2.2.2 How Should We Fill p_j ? Three Naive Ideas

(slides p. 14)

Idea	What p_j looks like	Why it fails
Constant vector	every entry equals j : $p_5 = [5, 5, \dots, 5]$	grows without bound, so for long sentences it swamps the word embedding; one number repeated 512 times carries no pattern
One-hot	1 at index j , 0 elsewhere	needs a fixed maximum length T ; all positions are equally far apart ($\sqrt{2}$, Section 2.2.5)
Learned embedding	a trainable vector per position $j = 0..T$	cannot produce vectors for positions beyond the longest training sentence

The slides sum up the common failure: *not suitable if the sentence length is dynamic*.

2.2.3 The Sinusoidal Encoding Function

(slides p. 15)

Hypothesis

Embed a **unique pattern of features** for each position j . The model will then learn to attend by *relative* position.

Sinusoidal positional encoding

For position j and frequency index $i = 0, 1, \dots, 255$ (with $d_{\text{model}} = 512$):

$$PE_{(j, 2i)} = \sin\left(\frac{j}{10000^{2i/d_{\text{model}}}}\right) \quad \text{even entry } 2i \quad (2.2.2)$$

$$PE_{(j, 2i+1)} = \cos\left(\frac{j}{10000^{2i/d_{\text{model}}}}\right) \quad \text{odd entry } 2i + 1 \quad (2.2.3)$$

Each frequency index i fills **two** neighbouring entries (one sin, one cos), so $256 \times 2 = 512$ entries. The vector $p_j = [PE_{(j,0)}, PE_{(j,1)}, \dots, PE_{(j,511)}]$.

Watch out: Reading the slide's formula

The slide writes both cases with “ $i = 0, 1, \dots, 255$ ” and says “sin if i is even, cos if i is odd”. These use i in two senses: the *frequency index* (0..255) inside the formula, and the *entry index* (0..511) when talking about even/odd. The unambiguous form is the one above. Entry $2i$ uses sin, entry $2i + 1$ uses cos, and **both use the same frequency** $\omega_i = 1/10000^{2i/512}$.

Reading the formula: frequencies and wavelengths

Write $\omega_i = 10000^{-2i/512}$. Then entry pair i is $(\sin \omega_i j, \cos \omega_i j)$, a point moving around a circle as j increases. Its wavelength (period in positions) is $\lambda_i = 2\pi/\omega_i = 2\pi \cdot 10000^{2i/512}$:

$$\lambda_0 = 2\pi \approx 6.3 \text{ positions}, \quad \lambda_{255} = 2\pi \cdot 10000^{510/512} \approx 60,600 \text{ positions}.$$

The wavelengths form a **geometric progression** from about 6 to about 60,000.

Intuition: A clock with 256 hands, or binary counting

Think of how binary numbers count: the last bit flips every step, the next every 2 steps, the next every 4. Reading all the bits together identifies the number uniquely. Sinusoidal PE

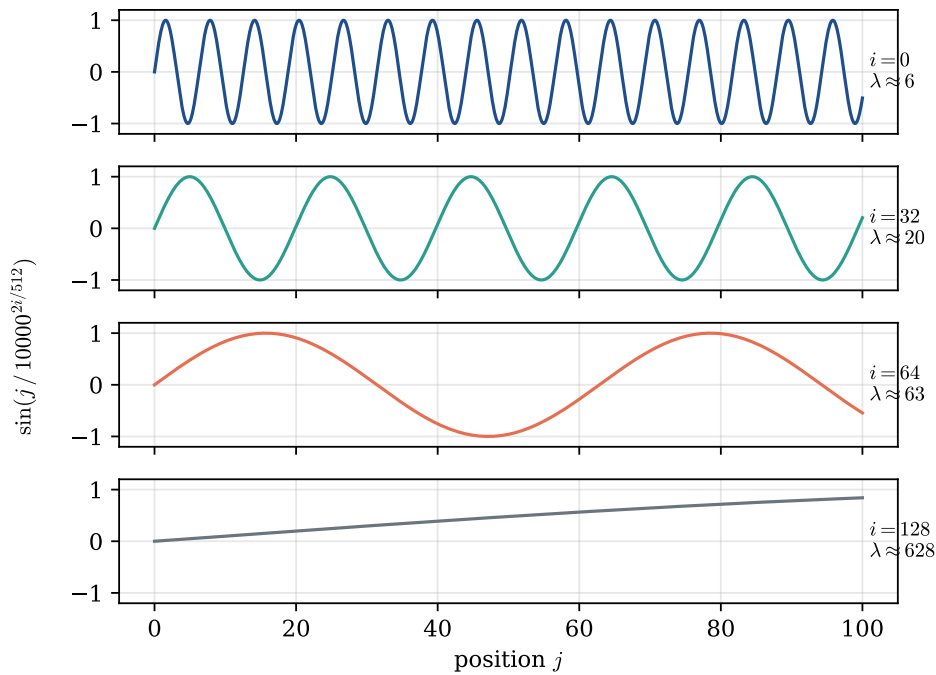


Figure 2.2.2: Four of the 256 sine channels as functions of position (wavelengths λ of about 6, 20, 63 and 628 positions). Low i oscillates quickly (it tells neighbouring positions apart). High i changes very slowly (it tells far-apart regions of the sentence apart).

does the same with smooth waves in place of bits. Fast channels act as the “seconds hand” (distinguishing j from $j+1$) and slow channels as the “hour hand” (distinguishing position 10 from position 500). Together they give every position a unique fingerprint, and the values always stay in $[-1, 1]$.

2.2.4 The Heat Map and the Special Case $j = 0$

(slides p. 15–17)

The slides evaluate $PE_{(j,i)}$ for $j = 0, \dots, 8$ and the first 64 entries and show the result as a heat map. Figure 2.2.3 reproduces it with the actual formula:

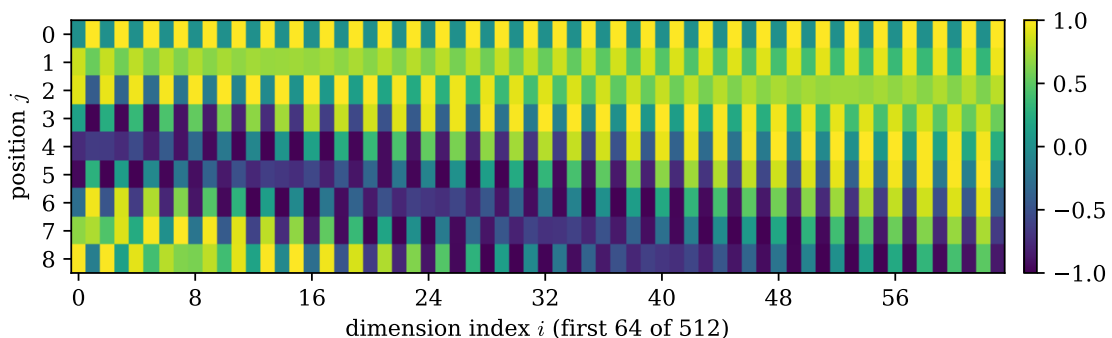


Figure 2.2.3: $PE_{(j,i)}$ for $j = 0..8$ (rows) and entries $i = 0..63$ (columns). Row 0 is a plain alternating 0/1 stripe. On the left, colours change quickly from row to row (fast frequencies). On the right, the columns are nearly constant down the rows (slow frequencies).

Derivation: Position $j = 0$

At $j = 0$ every argument is $0/10000^{2i/512} = 0$, so

$$PE_{(0,2i)} = \sin 0 = 0, \quad PE_{(0,2i+1)} = \cos 0 = 1 \implies p_0 = [0, 1, 0, 1, \dots, 0, 1]^T \in \mathbb{R}^{512}.$$

This alternating 0/1 vector is added to the **first word's embedding in every sentence**. Position 0 always gets the same “tag”. The same holds for every position j : the encoding depends only on j , never on the sentence.

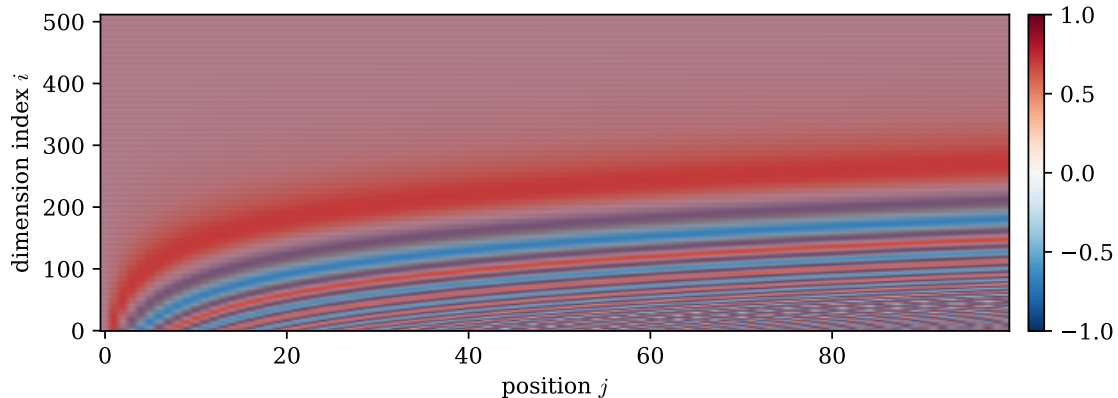


Figure 2.2.4: The full picture: all 512 entries (vertical) for positions 0 to 99 (horizontal). Only the bottom band (small i) changes quickly. Most of the upper entries barely move over 100 positions, which leaves room for much longer sequences.

2.2.5 Distance Properties

(slides p. 18–19)

What we want

For the sentence *I Enjoyed the film transformer*, the distances in word positions are:

	I	Enjoyed	the	film	transformer
I	0	1	2	3	4
Enjoyed	1	0	1	2	3
the	2	1	0	1	2
film	3	2	1	0	1
transformer	4	3	2	1	0

In every row, distance grows as we move left or right away from the 0, and it is symmetric around the diagonal. A good encoding should behave the same way: $\|p_j - p_k\|$ should *grow with* $|j - k|$.

Does the sinusoidal encoding satisfy this?

Derivation: The distance depends only on the offset $|j - k|$

Take one frequency pair i . Using $\sin^2 + \cos^2 = 1$ and $\cos a \cos b + \sin a \sin b = \cos(a - b)$:

$$\begin{aligned} (\sin \omega_i j - \sin \omega_i k)^2 + (\cos \omega_i j - \cos \omega_i k)^2 &= 2 - 2(\sin \omega_i j \sin \omega_i k + \cos \omega_i j \cos \omega_i k) \\ &= 2 - 2\cos(\omega_i(j - k)). \end{aligned}$$

Summing over the 256 pairs:

$$\|p_j - p_k\|^2 = \sum_{i=0}^{255} \left[2 - 2 \cos(\omega_i (j - k)) \right] \quad (2.2.4)$$

This depends **only on** $j - k$, and is symmetric because $\cos$ is even. So the distance matrix has the same structure as the table above: constant along each diagonal and symmetric. For small offsets each term grows with $|j - k|$, so nearby positions are close and farther ones are farther.

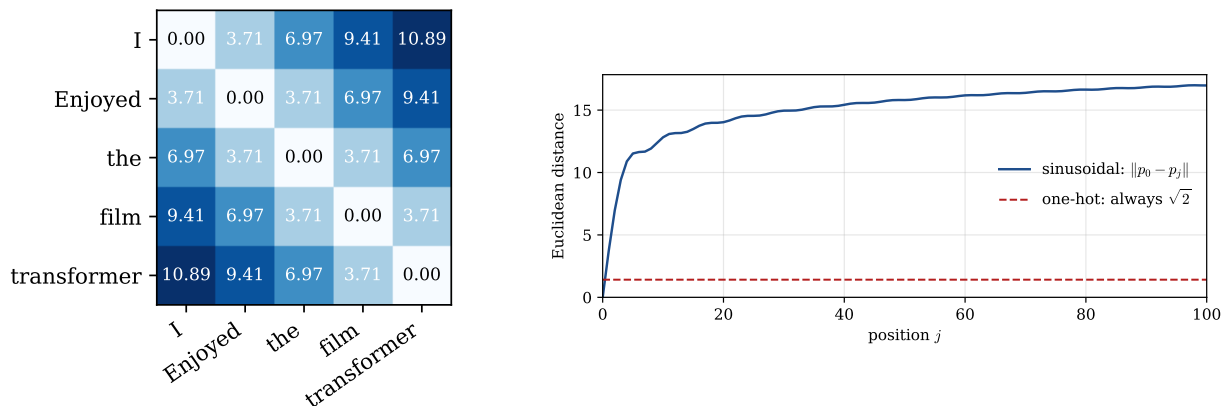


Figure 2.2.5: Left: actual Euclidean distances $\|p_j - p_k\|$ for the 5-word sentence. They have the same pattern as the word-distance table: 3.71 for neighbours, 6.97 for distance 2, and so on. Right: $\|p_0 - p_j\|$ for $j = 0..100$ (the slides' plot). The trend is clearly upward, with small wiggles. One-hot vectors give a flat $\sqrt{2}$ for every pair.

Watch out: “Increasing”, not “strictly monotone”

Look closely at the right-hand plot. The curve rises overall but **wiggles**: each $\cos$ term oscillates, so the sum is not strictly increasing. The correct exam statement is: *the PE function approximately satisfies the property; the distance grows with the gap between positions, and it depends only on that gap.*

Why one-hot encoding fails

For one-hot vectors u_j and u_k with $j \neq k$:

$$\|u_j - u_k\|^2 = \underbrace{1^2}_{\text{entry } j} + \underbrace{(-1)^2}_{\text{entry } k} = 2 \implies \|u_j - u_k\| = \sqrt{2} \text{ for every pair.}$$

Positions 1 and 2 are exactly as far apart as positions 1 and 1000, so one-hot encoding carries **no notion of nearness**.

2.2.6 Relative Position as a Linear Map

The hypothesis says the model will “learn to attend by relative position”. Here is the reason that is possible.

Derivation: Shifting by k is a rotation

For one frequency ω , the angle-addition formulas give

$$\begin{bmatrix} \sin \omega(j+k) \\ \cos \omega(j+k) \end{bmatrix} = \underbrace{\begin{bmatrix} \cos \omega k & \sin \omega k \\ -\sin \omega k & \cos \omega k \end{bmatrix}}_{R_k \text{ (depends only on } k\text{)}} \begin{bmatrix} \sin \omega j \\ \cos \omega j \end{bmatrix}.$$

Stacking all 256 pairs gives a block-diagonal matrix $\mathcal{R}_k$ with $p_{j+k} = \mathcal{R}_k p_j$ for *every* j . So “ k positions later” is one fixed linear map, and linear maps are exactly what attention’s W_Q and W_K can learn.

Intuition: Why both sine *and* cosine

With only sin, knowing $\sin(\omega j)$ is not enough to compute $\sin(\omega(j+k))$: the formula also needs $\cos(\omega j)$. The (sin, cos) pair is the smallest bundle of numbers that makes “shift by k ” a linear operation.

Beyond the slides: what modern LLMs use

The sinusoidal encoding is added once at the input. The slides also note it “can be parameterised”, i.e. learned (as in BERT and GPT-2). Most current LLMs (LLaMA, Qwen, Mistral) use **RoPE** (rotary position embedding). RoPE takes the rotation idea above and applies the rotation R_j directly to q and k inside every attention layer, so that $q_i \cdot k_j$ depends only on $i - j$. If you understand this section, you already understand most of RoPE.

Module 2.3

Training the Transformer

(slides p. 20–29, lecture Module 16.4)

2.3.1 How Deep Is the Transformer?

(slides p. 21–22)

For a rough comparison with ordinary deep networks, count each attention sublayer as one “attention layer” and each FFN as **two hidden layers** (it has two weight matrices, W_1 and W_2):

- encoder layer: 1 attention layer + 2 hidden layers = 3;
- decoder layer: 2 attention layers (masked self and cross) + 2 hidden layers = 4;
- total: $6 \times 3 + 6 \times 4 = 18 + 24 = \mathbf{42}$ layers.

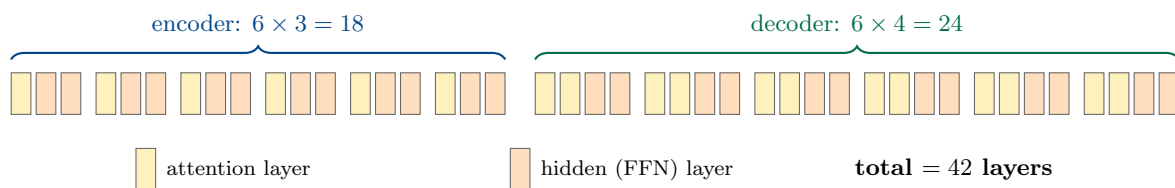


Figure 2.3.1: Counting the transformer as a plain deep network, as on the slides: 42 layers.

A 42-layer network raises the two classic deep-learning problems. The slides pair each with its answer:

Question	Answer
How do we ensure gradient flow across the network?	Residual connections (Section 2.3.5)
How do we speed up the training process?	Normalization (Sections 2.3.2–2.3.4)

2.3.2 Batch Normalization: Recap

(slides p. 23–24)

Consider layer l . Let x_i^j be the activation of **neuron** i for **training sample** j in a mini-batch of size m . Picture an *accumulator*: a table with one row per neuron and one column per sample. Batch norm works along a **row**. It normalises one neuron using its values across the batch:

Batch normalization (neuron i , layer l)

$$\mu_i = \frac{1}{m} \sum_{j=1}^m x_i^j \quad \text{mean of neuron } i \text{ over the batch} \quad (2.3.1)$$

$$\sigma_i^2 = \frac{1}{m} \sum_{j=1}^m (x_i^j - \mu_i)^2 \quad \text{variance of neuron } i \text{ over the batch} \quad (2.3.2)$$

$$\hat{x}_i = \frac{x_i - \mu_i}{\sqrt{\sigma_i^2 + \epsilon}} \quad \text{normalise} \quad (2.3.3)$$

$$\hat{y}_i = \gamma \hat{x}_i + \beta \quad \text{learnable scale } \gamma \text{ and shift } \beta \quad (2.3.4)$$

Every neuron i gets its own (μ_i, σ_i) . There are H pairs for a layer of H neurons.

The slides point out that three variables are involved: the layer l , the neuron i and the sample j . Draw them as three axes of a cube. For a fixed layer, batch norm takes a slice along j (one neuron, all samples). Layer norm, next, takes a slice along i (one sample, all neurons).

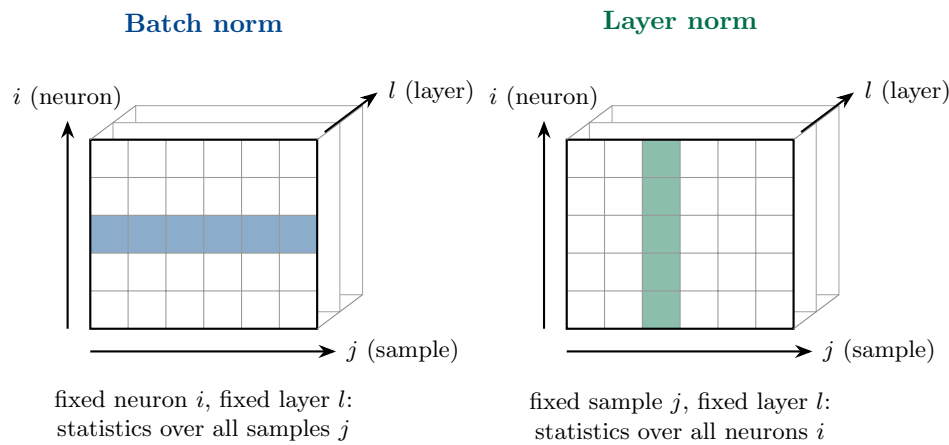


Figure 2.3.2: The (l, i, j) cube from the slides. The front face is one layer's accumulator (neurons $\times$ samples). Batch norm normalises a **row** (blue). Layer norm normalises a **column** (green).

2.3.3 Limitations of Batch Normalization

(slides p. 25)

Can we apply batch norm to transformers? **Of course, yes**, but it has limitations:

1. **Depends on m .** μ_i and σ_i^2 are *estimates* from m samples. Small m gives noisy estimates and high error.
2. **Useless at $m = 1$.** With one sample, $\mu_i = x_i$ and $\sigma_i = 0$, so $\hat{x}_i = 0/\sqrt{\epsilon} = 0$ for every neuron. It makes no difference (the slides' words), because all information is wiped out.
3. **Worse results in NLP.** It was found empirically that naive batch norm degrades performance on NLP tasks. A systematic study confirmed this and proposed a modified batch norm called **PowerNorm**.

Intuition: Why batch statistics suit text badly

In a vision batch, “pixel feature 17” means roughly the same thing in every image. In a text batch, sentences have different lengths and are padded, so the pad positions pollute the statistics. The same position also holds a noun in one sentence and a verb in another, so the batch statistics jump around from one batch to the next. Statistics *within* one token's vector are far more stable.

2.3.4 Layer Normalization

(slides p. 26)

Layer normalization (sample fixed, layer l with H neurons)

$$\mu_l = \frac{1}{H} \sum_{i=1}^H x_i \quad \text{mean over the neurons of this layer} \quad (2.3.5)$$

$$\sigma_l = \sqrt{\frac{1}{H} \sum_{i=1}^H (x_i - \mu_l)^2} \quad \text{std over the neurons} \quad (2.3.6)$$

$$\hat{x}_i = \frac{x_i - \mu_l}{\sqrt{\sigma_l^2 + \epsilon}} \quad \forall i \quad \text{one } (\mu, \sigma) \text{ shared by all neurons} \quad (2.3.7)$$

$$\hat{y}_i = \gamma_i \hat{x}_i + \beta_i \quad \gamma, \beta \in \mathbb{R}^H \text{ learnable} \quad (2.3.8)$$

In the transformer $H = d_{\text{model}} = 512$: each token's 512-vector is normalised on its own.

The computation is simple: average the outputs of the hidden units in the layer. So the normalisation does not depend on the number of samples in a batch. This allows batch size 1 (needed in RNNs, for example, and at inference time).

Derivation: BN vs LN on the same numbers

Accumulator for $H = 4$ neurons and $m = 3$ samples. **Rows** of the table below are samples (so for batch norm read down a column):

	x_1	x_2	x_3	x_4
sample 1	2	4	6	8
sample 2	4	4	2	0
sample 3	6	1	1	4

Batch norm (per neuron, down each column): $\mu = [4, 3, 3, 4]$, $\sigma^2 = [2.67, 2, 4.67, 10.67]$. For neuron 1: $(2 - 4)/\sqrt{2.67} = -1.225$, $(4 - 4)/\cdot = 0$, $(6 - 4)/\sqrt{2.67} = 1.225$.

Layer norm (per sample, across each row): sample 1 has $\mu = 5$, $\sigma^2 = 5$, which gives $[-1.342, -0.447, 0.447, 1.342]$. Sample 2 has $\mu = 2.5$, $\sigma^2 = 2.75$, which gives $[0.905, 0.905, -0.302, -1.508]$.

Delete samples 2 and 3 and batch norm collapses (everything becomes 0). Layer norm for sample 1 is **unchanged**, because it never looked at the other samples.

	Batch norm	Layer norm
Statistics over	samples in the batch (per neuron)	neurons in the layer (per sample)
Number of (μ, σ) pairs	one per neuron	one per sample
Works with $m = 1$?	no ($\sigma = 0$)	yes
Train vs test behaviour	differs (running averages at test)	identical
Used in transformers?	no	yes ("Add & Norm")

2.3.5 Residual Connections

(slides p. 27–28)

Every sublayer (attention, cross-attention, FFN) is wrapped as

$$\text{output} = \text{LayerNorm}(x + \text{Sublayer}(x)). \quad (2.3.9)$$

The " $x +$ " is the **residual (skip) connection**, and LayerNorm is the norm. Together they are the **Add & Norm** block drawn in every transformer diagram.

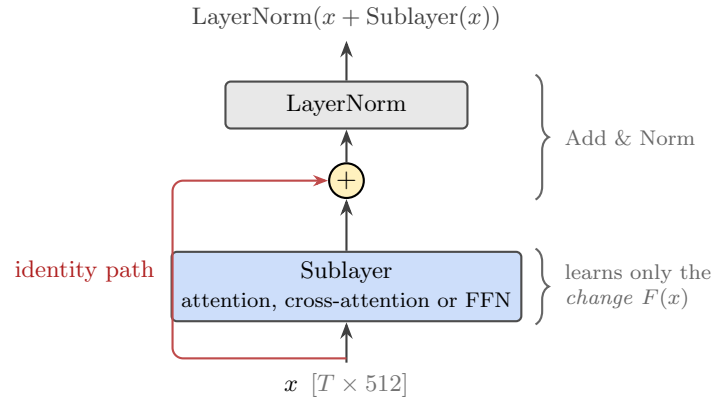


Figure 2.3.3: The Add & Norm wrapper used around every sublayer.

Derivation: Why the skip connection keeps gradients alive

Ignore the LayerNorm for a moment and write each block as $x_{k+1} = x_k + F_k(x_k)$. By the chain rule

$$\frac{\partial x_{k+1}}{\partial x_k} = I + \frac{\partial F_k}{\partial x_k} \implies \frac{\partial \mathcal{L}}{\partial x_k} = \frac{\partial \mathcal{L}}{\partial x_L} \prod_{m=k}^{L-1} \left(I + \frac{\partial F_m}{\partial x_m} \right).$$

Multiply out the product and one term is exactly I : a direct path from the loss to layer k that passes through *no* weights. **Without** skips the product is $\prod \partial F_m / \partial x_m$. If each factor has size about 0.5, then over 42 layers $0.5^{42} \approx 2 \times 10^{-13}$, and the gradient has vanished. With skips, the “1 +” in each factor keeps a path open no matter how small $\partial F / \partial x$ gets.

Intuition: Learn the edit, not the whole document

With a skip connection each sublayer only has to learn a *correction* $F(x)$ to its input. If a layer has nothing useful to add, it can output $F(x) \approx 0$ and pass x through unchanged. A deep stack therefore starts out close to the identity function, which is easy to optimise.

2.3.6 The Complete Layers

(slides p. 27–28)

The slides’ final rule: *add a residual connection and layer norm block after every multi-head attention, feed-forward network and cross-attention block.*

Encoder layer (2 sublayers) and decoder layer (3 sublayers)

Encoder:

$$\begin{aligned} z &= \text{LayerNorm}(x + \text{MHA}(x)) \\ \text{out} &= \text{LayerNorm}(z + \text{FFN}(z)) \end{aligned}$$

Decoder:

$$\begin{aligned} s &= \text{LayerNorm}(y + \text{MaskedMHA}(y)) \\ c &= \text{LayerNorm}(s + \text{CrossMHA}(s, E)) \\ \text{out} &= \text{LayerNorm}(c + \text{FFN}(c)) \end{aligned}$$

2.3.7 The Complete Architecture: Final Remarks

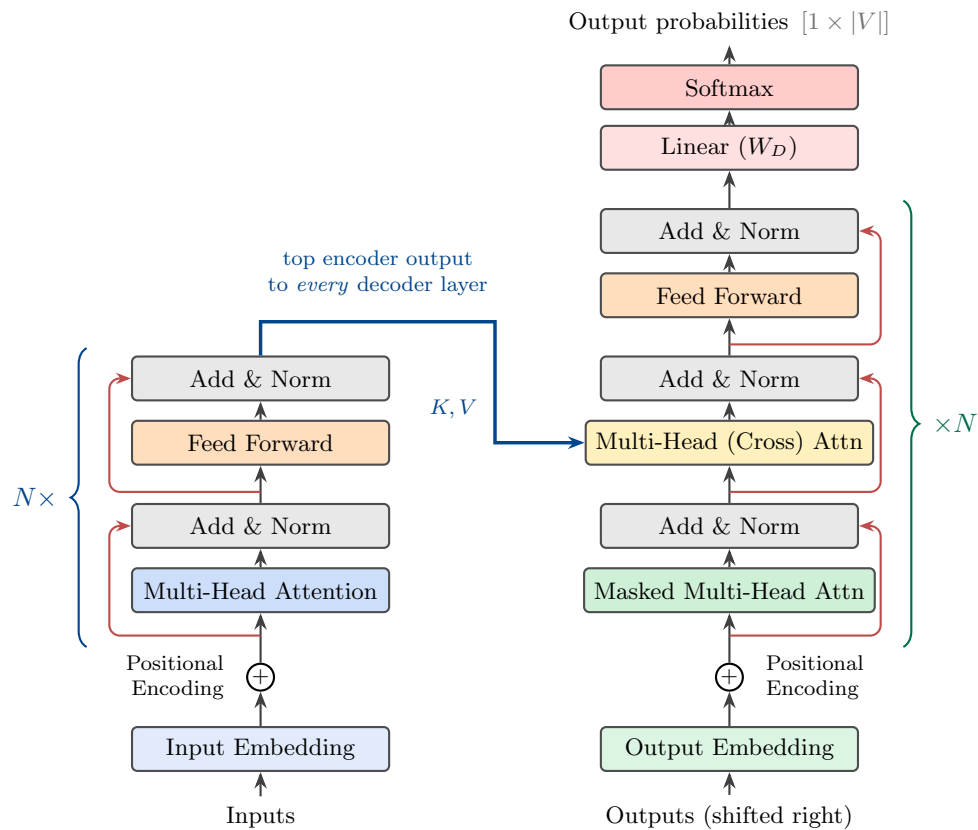


Figure 2.3.4: The complete transformer (“Attention Is All You Need”), redrawn from the slides. Red arrows are residual connections, and each one ends in an Add & Norm block.

(slides p. 29)

The last slide adds three details that are easy to overlook (and easy to ask in an exam):

1. **Input embeddings are learned during training.** No pretrained embeddings such as Word2Vec are used. This is an additional set of weights in the model.
2. **The positional encoding is added to the input embedding**, and the function *can also be parameterised* (learned) in place of the fixed sinusoid.
3. **The output of the top encoder layer is fed to all decoder layers** to compute multi-head cross-attention.

Tracing shapes through the whole model

Row convention, one sentence pair, T_s source and T_t target tokens:

Stage	Operation	Output shape
Source tokens	token ids	T_s
Embedding + PE	lookup in 37000×512 , add p_j	$T_s \times 512$
Encoder layer $\times 6$	MHA, Add&Norm, FFN, Add&Norm	$T_s \times 512$
inside MHA, per head	$QK^T/\sqrt{64}$	$T_s \times T_s$
inside FFN	expand, ReLU, project	$T_s \times 2048 \rightarrow T_s \times 512$
Encoder output E		$T_s \times 512$
Target tokens (shifted)	$\langle \text{Go} \rangle y_1 \dots y_{T_t-1}$	T_t
Embedding + PE		$T_t \times 512$
masked self-attn, per head	$QK^T/8 + M$	$T_t \times T_t$
cross-attn, per head	$Q_{\text{dec}}K_{\text{enc}}^T/8$	$T_t \times T_s$
Decoder layer $\times 6$		$T_t \times 512$
Linear W_D	$\times (512 \times 37000)$	$T_t \times 37000$
Softmax	row-wise	$T_t \times 37000$, rows sum to 1

Total parameter count

Component	Exact count	Rounded
One MHA sublayer ($4 \times 512^2 + \text{biases}$)	1,050,624	1M
One FFN	2,099,712	2M
One encoder layer (MHA + FFN + 2 LN)	3,152,384	3M
One decoder layer (2 MHA + FFN + 3 LN)	4,204,032	4M
Encoder, 6 layers	18,914,304	19M
Decoder, 6 layers	25,224,192	25M
Shared embedding = W_D (37000×512)	18,944,000	19M
Total (with weight sharing)	63,082,496	$\approx 63\text{M}$
Total if the 3 embedding matrices were separate	100,970,496	$\approx 101\text{M}$

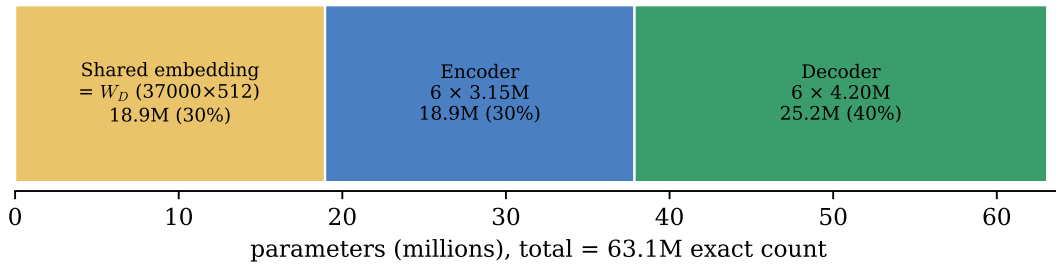


Figure 2.3.5: Where the parameters live. The single vocabulary matrix is about as large as the whole 6-layer encoder.

Watch out: “65M” vs 63M

The slides (and the paper) quote $\approx 65\text{M}$ for the base model. A careful count under the stated sizes gives $\approx 63\text{M}$. The small gap comes from implementation details such as the exact vocabulary size. For an exam, quote the slides: $\approx 4\text{M}$ per decoder layer, $\approx 19\text{M}$ for W_D , $\approx 65\text{M}$ total. If asked to *compute*, show the working as in the table.

Beyond the slides: Post-LN vs Pre-LN

The original transformer (and these slides) apply the norm *after* the addition: $\text{LayerNorm}(x + F(x))$, called **Post-LN**. Almost all modern LLMs use **Pre-LN**: $x + F(\text{LayerNorm}(x))$. This leaves the residual path completely clean (pure identity), so very deep models train more stably with less learning-rate warm-up. Many also replace LayerNorm with **RMSNorm**, which skips the mean subtraction and is cheaper.

One-Page Revision Sheet

Decoder (2.1)

- Output per position: $1 \times |V|$ distribution (e.g. 1×37000).
- $N = 6$ layers, each with 3 sublayers: masked self-attn $\rightarrow$ cross-attn $\rightarrow$ FFN.
- Teacher forcing: training input = ground truth shifted right ($\langle \text{Go} \rangle$ first). Avoids error accumulation; training-only.
- Masked attention: $Z = \text{softmax}(A + M)V^T$, $A = Q^T K / \sqrt{d_k}$.
- M : 0 on/below the diagonal, $-\infty$ above. Added *before* the softmax.
- Encoder masks too, but for *padding*.
- Cross-attn: $Q_2 = W_{Q_2}S$ (decoder), $K_2 = W_{K_2}E$, $V_2 = W_{V_2}E$ (encoder). Weights are $T_t \times T_s$. No causal mask.
- E from the *top* encoder layer goes to *all* 6 decoder layers.
- Params per decoder layer: $1\text{M} + 1\text{M} + 2\text{M} \approx 4\text{M}$.
- FFN = $2(512 \cdot 2048) + 2048 + 512 \approx 2\text{M}$.
- W_D : $512 \times |V| \approx 19\text{M}$ of $\approx 65\text{M}$.
- Inference is autoregressive: one decoder pass per token.

Positional encoding (2.2)

- Self-attention ignores order, so add p_j : $h'_j = h_j + p_j \in \mathbb{R}^{512}$.
- Rejected: constant j (unbounded), one-hot (distance always $\sqrt{2}$), learned (fixed max length).
- $PE_{(j,2i)} = \sin(j/10000^{2i/512})$, $PE_{(j,2i+1)} = \cos(\cdot)$, $i = 0..255$.
- Wavelengths $2\pi \rightarrow \approx 2\pi \cdot 10^4$, geometric.
- $p_0 = [0, 1, 0, 1, \dots]$, added to the first word of every sentence.
- $\|p_j - p_k\|^2 = \sum_i [2 - 2\cos(\omega_i(j-k))]$: depends only on $|j-k|$ and grows with it (with small wiggles).
- $p_{j+k} = \mathcal{R}_k p_j$: a shift is a fixed rotation, which is what enables attention by relative position.

Training (2.3)

- Depth: encoder $6 \times (1 + 2)$ + decoder $6 \times (2 + 2)$ = 42 layers.
- Gradient flow $\rightarrow$ residual connections. Speed $\rightarrow$ normalization.
- BN: statistics over the batch, per neuron i : $\mu_i = \frac{1}{m} \sum_j x_i^j$.
- BN limits: depends on m ; useless at $m = 1$ ($\mu_i = x_i$, $\sigma_i = 0$); degrades NLP results (PowerNorm).
- LN: statistics over neurons, per sample: $\mu_l = \frac{1}{H} \sum_i x_i$. Batch-independent.
- Add & Norm: $\text{LayerNorm}(x + \text{Sublayer}(x))$ after every attention, cross-attention and FFN.
- Skip gives $\partial x_{k+1} / \partial x_k = I + \partial F / \partial x$, an identity gradient path.
- Encoder: 2 Add&Norm per layer. Decoder: 3.
- Embeddings learned from scratch (no Word2Vec). PE can be learned too.

Shapes to know

- $d_{\text{model}} = 512$, $h = 8$, $d_k = d_v = 64$, $d_{ff} = 2048$, $N = 6$.
- Self-attn weights $T \times T$. Cross-attn weights $T_t \times T_s$.
- Every sublayer maps $T \times 512 \rightarrow T \times 512$.
- Logits $T_t \times |V|$.

Exam traps

- Mask is added to the *scores*, not multiplied into the weights.
- Q in cross-attention comes from the decoder, not the encoder.
- LN normalises across features, not across the batch.
- Teacher forcing is *not* used at inference.
- BN at $m = 1$ outputs zeros (then β), not NaN, thanks to ϵ .
- One-hot PE distance is $\sqrt{2}$, not 1 or 2.
- Two neighbouring PE entries share one frequency.

Practice

Try each problem before reading its solution. Problems are tagged with the module they test. All numbers have been checked by direct computation.

Part A: Numerical Problems with Worked Solutions

A1 · Output dimension and W_D (2.1)

An English to French transformer has $|V| = 42,000$ and $d_{\text{model}} = 512$. (a) What does the decoder output at each position? (b) How many parameters are in W_D , with and without a bias? (c) For a 9-token target, what is the shape of the full output?

Solution (a) A probability distribution over the French vocabulary: $1 \times 42,000$.
 (b) $W_D \in \mathbb{R}^{512 \times 42000}$: $512 \times 42,000 = 21,504,000 \approx 21.5\text{M}$. A bias adds 42,000, for 21,546,000.
 (c) $9 \times 42,000$. Each row sums to 1.

A2 · Applying the causal mask (2.1)

For $T = 4$ the (already scaled) scores are $A = \begin{bmatrix} 2 & 3 & 1 & 5 \\ 4 & 1 & 2 & 3 \\ 1 & 5 & 3 & 2 \\ 2 & 4 & 1 & 6 \end{bmatrix}$. Write M and $A + M$, and compute the attention weights for every row (rows numbered 1 to 4).

Solution

$$M = \begin{bmatrix} 0 & -\infty & -\infty & -\infty \\ 0 & 0 & -\infty & -\infty \\ 0 & 0 & 0 & -\infty \\ 0 & 0 & 0 & 0 \end{bmatrix}, \quad A + M = \begin{bmatrix} 2 & -\infty & -\infty & -\infty \\ 4 & 1 & -\infty & -\infty \\ 1 & 5 & 3 & -\infty \\ 2 & 4 & 1 & 6 \end{bmatrix}.$$

Row-wise softmax, with $e^{-\infty} = 0$:

- Row 1: only one allowed entry, so $[1, 0, 0, 0]$.
- Row 2: $e^4 = 54.60$, $e^1 = 2.72$, sum 57.32, so $[0.953, 0.047, 0, 0]$.
- Row 3: $e^1 = 2.72$, $e^5 = 148.41$, $e^3 = 20.09$, sum 171.22, so $[0.016, 0.867, 0.117, 0]$.
- Row 4: $e^2 = 7.39$, $e^4 = 54.60$, $e^1 = 2.72$, $e^6 = 403.43$, sum 468.13, so $[0.016, 0.117, 0.006, 0.862]$.

Check: every row sums to 1, and everything above the diagonal is exactly 0. Note that row 4 sees all four tokens, but row 1 sees only itself, however large $A_{14} = 5$ is.

A3 · A tiny cross-attention (2.1)

Row convention, $d_k = 2$. The decoder gives $Q = \begin{bmatrix} 1 & 0 \\ 0 & 2 \end{bmatrix}$ ($T_t = 2$). The encoder gives $K = \begin{bmatrix} 1 & 1 \\ 2 & 0 \\ 0 & 1 \end{bmatrix}$ and $V = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}$ ($T_s = 3$). Compute $Z = \text{softmax}(QK^T/\sqrt{d_k})V$ and state every shape.

Solution $QK^T = \begin{bmatrix} 1 & 2 & 0 \\ 2 & 0 & 2 \end{bmatrix} [2 \times 3]$, divided by $\sqrt{2}$: $\begin{bmatrix} 0.707 & 1.414 & 0 \\ 1.414 & 0 & 1.414 \end{bmatrix}$.

- Row 1: $e^{0.707} = 2.028$, $e^{1.414} = 4.113$, $e^0 = 1$, sum 7.141, so $\alpha_1 = [0.284, 0.576, 0.140]$.
- Row 2: 4.113, 1, 4.113, sum 9.226, so $\alpha_2 = [0.446, 0.108, 0.446]$.

$Z = \alpha V$: row 1 = $0.284[1, 0] + 0.576[0, 1] + 0.140[1, 1] = [0.424, 0.716]$; row 2 = $0.446[1, 0] + 0.108[0, 1] + 0.446[1, 1] = [0.892, 0.554]$.

Shapes: weights 2×3 ($T_t \times T_s$, rectangular), output 2×2 ($T_t \times d_v$). No mask is used, since every target position may see every source word.

A4 · Computing positional encodings (2.2)

With $d_{\text{model}} = 512$, compute entries 0 to 3 of p_1 and p_2 .

Solution Entries 0 and 1 use $i = 0$: $\omega_0 = 1$. Entries 2 and 3 use $i = 1$: $\omega_1 = 10000^{-2/512} = e^{-(2/512) \ln 10000} = e^{-0.03598} = 0.96466$.

	entry 0: $\sin(j)$	entry 1: $\cos(j)$	entry 2: $\sin(0.96466j)$	entry 3: $\cos(0.96466j)$
p_1	0.8415	0.5403	0.8219	0.5697
p_2	0.9093	-0.4161	0.9364	-0.3509

Entries 2 and 3 are close to entries 0 and 1 because ω_1 is close to ω_0 . The high entries (e.g. 510, 511) use $\omega_{255} \approx 10^{-4}$, so they stay near $(0, 1)$ for all small j .

A5 · Distances in a toy PE (2.2)

Use a toy $d_{\text{model}} = 4$ (so $i \in \{0, 1\}$, $\omega_0 = 1$, $\omega_1 = 10000^{-2/4} = 0.01$). (a) Write $p_0, \dots, p_3$. (b) Compute $\|p_0 - p_1\|$, $\|p_1 - p_2\|$, $\|p_0 - p_2\|$, $\|p_0 - p_3\|$ with the closed formula. (c) What pattern do you see?

Solution (a) $p_j = [\sin j, \cos j, \sin 0.01j, \cos 0.01j]$: $p_0 = [0, 1, 0, 1]$, $p_1 = [0.841, 0.540, 0.010, 1.000]$, $p_2 = [0.909, -0.416, 0.020, 1.000]$, $p_3 = [0.141, -0.990, 0.030, 1.000]$.

(b) $\|p_j - p_k\|^2 = \sum_i [2 - 2 \cos(\omega_i \Delta)]$ with $\Delta = j - k$:

$$\Delta = 1 : (2 - 2 \cos 1) + (2 - 2 \cos 0.01) = 0.91940 + 0.00010 = 0.91950 \Rightarrow 0.959$$

$$\Delta = 2 : (2 - 2 \cos 2) + (2 - 2 \cos 0.02) = 2.83229 + 0.00040 = 2.83269 \Rightarrow 1.683$$

$$\Delta = 3 : \Rightarrow 1.995$$

So $\|p_0 - p_1\| = \|p_1 - p_2\| = 0.959$, $\|p_0 - p_2\| = 1.683$ and $\|p_0 - p_3\| = 1.995$.

(c) Equal offsets give equal distances (only $|j - k|$ matters), and the distance grows with the offset. Compare one-hot, where all three would be $\sqrt{2} = 1.414$.

A6 · Parameter count, base model (2.1, 2.3)

$d_{\text{model}} = 512$, $h = 8$, $d_k = d_v = 64$, $d_{ff} = 2048$, $N = 6$, $|V| = 37,000$ (shared embeddings). Count (a) one MHA sublayer, (b) one FFN, (c) one encoder layer, (d) one decoder layer, (e) the total. Include biases and LayerNorm.

Solution (a) W_Q, W_K, W_V for 8 heads: $3 \cdot 8 \cdot 512 \cdot 64 = 786,432$. W_O : 262,144. Biases $4 \times 512 = 2,048$. Total **1,050,624**.

(b) $2 \cdot 512 \cdot 2048 + 2048 + 512 = \mathbf{2,099,712}$.

(c) MHA + FFN + 2 LayerNorms (2×1024) = **3,152,384** ≈ 3 M.

(d) 2 MHA + FFN + 3 LayerNorms = $2,101,248 + 2,099,712 + 3,072 = \mathbf{4,204,032} \approx 4$ M.

(e) Embedding $37,000 \cdot 512 = 18,944,000$. Encoder $6 \times (c) = 18,914,304$. Decoder $6 \times (d) = 25,224,192$. Total = **63,082,496** ≈ 63 M. The slides and paper quote ≈ 65 M (see Section 2.3.7).

A7 · Parameter count, a smaller model (2.1, 2.3)

Repeat A6 for $d_{\text{model}} = 256$, $h = 4$, $d_{\text{ff}} = 1024$, $N = 4$, $|V| = 16,000$ (shared embeddings). Which part dominates?

Solution MHA: $4 \cdot 256^2 + 4 \cdot 256 = 263,168$. FFN: $2 \cdot 256 \cdot 1024 + 1024 + 256 = 525,568$.

Encoder layer: $263,168 + 525,568 + 1,024 = 789,760$. Decoder layer: $2 \cdot 263,168 + 525,568 + 1,536 = 1,053,440$.

Embedding: $16,000 \cdot 256 = 4,096,000$.

Total = $4,096,000 + 4(789,760) + 4(1,053,440) = \mathbf{11,468,800} \approx 11.5$ M. The embedding alone is 36% of the model. Small models are dominated by the vocabulary matrix. Note also that h does not change the count: $h \cdot d_k = d_{\text{model}}$ however the heads are split.

A8 · Layer norm with learned γ, β (2.3)

One token has activations $x = [2, 4, 6, 8]$. Compute the LayerNorm output with $\gamma = 2$, $\beta = 1$, $\epsilon = 10^{-5}$.

Solution $\mu = 5$. $\sigma^2 = \frac{9+1+1+9}{4} = 5$. $\sqrt{5 + 10^{-5}} = 2.2361$.

$\hat{x} = [-3, -1, 1, 3]/2.2361 = [-1.342, -0.447, 0.447, 1.342]$ (mean 0, variance 1).

$\hat{y} = 2\hat{x} + 1 = [-1.683, 0.106, 1.894, 3.683]$. The output now has mean $\beta = 1$ and standard deviation $\gamma = 2$. The learnable parameters let the network undo the normalisation if that helps.

A9 · Batch norm vs layer norm; batch size 1 (2.3)

Using the 3×4 accumulator from Section 2.3.4 (rows = samples: $[2, 4, 6, 8]$, $[4, 4, 2, 0]$, $[6, 1, 1, 4]$): (a) compute the BN output for neuron 3; (b) compute the LN output for sample 3; (c) what does BN output if only sample 1 is in the batch?

Solution (a) Neuron 3 values: 6, 2, 1. $\mu_3 = 3$, $\sigma_3^2 = \frac{9+1+4}{3} = 4.667$, $\sigma_3 = 2.160$. Output $[1.389, -0.463, -0.926]$.

(b) Sample 3: $[6, 1, 1, 4]$. $\mu = 3$, $\sigma^2 = \frac{9+4+4+1}{4} = 4.5$, $\sigma = 2.121$. Output $[1.414, -0.943, -0.943, 0.471]$.

(c) With $m = 1$: $\mu_i = x_i$ and $\sigma_i = 0$, so $\hat{x}_i = 0/\sqrt{\epsilon} = 0$ for every neuron. After scale and shift the output is just β everywhere. All information about the input is lost. LN on the same single sample gives $[-1.342, -0.447, 0.447, 1.342]$, which is unaffected.

A10 · Gradients with and without skips (2.3)

Model each block as a scalar map with local derivative $f' = 0.5$. (a) What factor multiplies the gradient reaching the first layer of a 12-layer and a 42-layer *plain* stack? (b) With residual blocks $x_{k+1} = x_k + F(x_k)$, why can the gradient not vanish this way?

Solution (a) $0.5^{12} = 2.4 \times 10^{-4}$ and $0.5^{42} = 2.3 \times 10^{-13}$. The first layers essentially stop learning.

(b) Each factor becomes $\partial x_{k+1} / \partial x_k = 1 + F'_k$. Expanding $\prod_k (1 + F'_k) = 1 + \sum_k F'_k + \sum_{k < m} F'_k F'_m + \dots$ always contains the constant term 1: a path that bypasses every sublayer. The gradient signal reaching the early layers can never be smaller than the signal carried by that identity path. (LayerNorm then keeps the forward activations well-scaled so the sum does not blow up either.)

A11 · Counting in the mask and in decoding (2.1, 2.3)

(a) For $T = 6$, how many entries of M are $-\infty$, and how many are 0? (b) How many “layers” deep is a transformer with $N = 12$ encoder and $N = 12$ decoder layers, by the slides’ counting? (c) To translate a sentence into a 20-token output (including $\langle \text{eos} \rangle$), how many encoder and decoder forward passes are needed at inference? And during training with teacher forcing?

Solution (a) Above the diagonal: $\frac{T(T-1)}{2} = 15$ entries of $-\infty$. On or below it: $\frac{T(T+1)}{2} = 21$ zeros. Check: $15 + 21 = 36 = 6^2$.

(b) $12 \times 3 + 12 \times 4 = 36 + 48 = 84$.

(c) Inference: **1** encoder pass and **20** decoder passes, one per generated token. Training: 1 encoder pass and **1** decoder pass, since all 20 positions are computed in parallel thanks to teacher forcing and the mask.

Part B: Subjective Questions with Model Answers

B1

Why is the decoder's self-attention masked but the encoder's is not?

Answer. The encoder builds a representation of a sentence that is fully known, so every word may use both its left and right context. The decoder *generates*. At inference, when producing word t , words $t+1, \dots$ do not exist yet. In training, teacher forcing feeds the whole target at once for parallelism. Without a mask, position t could attend to position $t+1$, which is the very word it must predict. It would learn to copy, reach near-zero training loss, and fail completely at inference. The causal mask ($-\infty$ above the diagonal, added before the softmax) gives each position exactly the view it will have at inference.

B2

Explain teacher forcing, its benefit, and its cost

Answer. With teacher forcing, the decoder receives the ground-truth previous token during training, not its own prediction. *Benefit:* a wrong prediction cannot corrupt later inputs, so errors do not accumulate through the chain rule $p(y) = \prod_t p(y_t \mid y_{<t})$. Every position gets a useful learning signal from the start, which speeds up training a lot, and all positions can be computed in one parallel pass. *Cost:* at inference the model is fed its own outputs, a situation it never saw in training (*exposure bias*). One early mistake can push it off the training distribution. Remedies include scheduled sampling, where own predictions are gradually mixed in during training.

B3

In cross-attention, why do queries come from the decoder and keys/values from the encoder?

Answer. Attention is a soft lookup: queries say *what I need*, keys say *what I contain*, and values are *what I hand over*. The decoder is the one generating, so at each step it needs information about the source; it asks the questions. The source words are what is being searched, so they supply keys and values. This also sets the output shape: one output per query, i.e. per target position ($T_t \times 512$), which is what the next decoder sublayer expects. Reversing the roles would produce one vector per *source* word, which is useless for predicting the next *target* word.

B4

Why does the transformer need positional encoding, and why were constant, one-hot and learned encodings rejected?

Answer. Attention scores depend only on token contents, and there is no recurrence, so re-ordering the input only reorders the output. The model cannot tell “dog bites man” from “man bites dog”. A position vector added to each embedding supplies the missing information. *Constant* ($p_j = [j, \dots, j]$) grows without bound and drowns the word embedding for long sentences. *One-hot* requires a fixed maximum length and makes all positions equidistant ($\sqrt{2}$), so there is no sense of nearness. *Learned* vectors exist only for positions seen in training, so they cannot handle longer sentences. All three fail when sentence length is dynamic.

B5

Why does sinusoidal PE use both sine and cosine, at many frequencies?

Answer. *Both functions:* the pair $(\sin \omega j, \cos \omega j)$ can be shifted by k positions with a fixed rotation R_k that does not depend on j . So “ k positions later” is a linear map that attention's projections can learn. With sin alone, $\sin(\omega(j+k))$ cannot be written as a linear function of

$\sin(\omega j)$. *Many frequencies*: a single fast wave repeats every 2π positions, so positions 0 and 6 look nearly alike ($6 \approx 2\pi$). Adding slower waves (wavelengths up to about 60,000) makes every position's full 512-vector unique, like the bits of a binary counter. Fast channels resolve neighbours and slow channels resolve distant positions.

B6

Compare batch normalization and layer normalization. Why does the transformer use LN?

Answer. Both subtract a mean, divide by a standard deviation, and apply learnable γ, β . BN computes the statistics for each neuron across the m samples in the batch. LN computes them for each sample across the H neurons of the layer. BN's estimates are noisy for small m and meaningless at $m = 1$ ($\mu = x, \sigma = 0$). It also behaves differently in training and at test, and empirically it degrades NLP results (variable-length, padded sequences make the batch statistics unstable; PowerNorm was proposed as a fix). LN is independent of the batch, identical at training and test time, works for one sentence at a time, and is cheap. So each sublayer is followed by Add & (Layer) Norm.

B7

What is “Add & Norm”? Explain how the residual part solves the depth problem.

Answer. Add & Norm computes $\text{LayerNorm}(x + \text{Sublayer}(x))$ after every attention, cross-attention and FFN block. By the slides' count the transformer is 42 layers deep. Without skips, backpropagation multiplies 42 Jacobians, and if each shrinks the signal the gradient vanishes (e.g. $0.5^{42} \approx 10^{-13}$). With $x + F(x)$ the Jacobian is $I + \partial F / \partial x$, so the product always contains an identity term: a clean path from the loss to every layer. Each sublayer only needs to learn a correction to its input, and the network starts near the identity function. LayerNorm keeps the activations on a stable scale, which speeds up training.

B8

The output layer W_D holds about 30% of all parameters. Why so large, and what can be done?

Answer. It maps every 512-dimensional vector to a score for *every* word in the vocabulary, so its size is $512 \times |V|$, which is 19M for $|V| = 37,000$. All other matrices are only 512×512 or 512×2048 . The count grows linearly with vocabulary size and does not depend on depth. Remedies: share (tie) this matrix with the input embeddings, as the original model does, and keep the vocabulary moderate with subword tokenisation.

Part C: Multiple-Choice Questions

Answers and explanations are on the pages after the questions, so you can attempt these first.

- C1.** For $|V| = 50,000$, the decoder output at one position has shape:
 (a) 1×512 (b) $1 \times 50,000$ (c) $512 \times 50,000$ (d) $50,000 \times 50,000$
- C2.** How many sublayers does one decoder layer have (not counting Add & Norm)?
 (a) 1 (b) 2 (c) 3 (d) 4
- C3.** During training with teacher forcing, the decoder input at position t is:
 (a) the ground-truth token y_{t-1}^* (b) the model's prediction $\hat{y}_{t-1}$ (c) the encoder output e_t
 (d) the ground-truth token y_t^*
- C4.** The causal mask M has:
 (a) 1 on/below the diagonal, 0 above (b) $-\infty$ on the diagonal (c) 0 above the diagonal, $-\infty$ below (d) 0 on/below the diagonal, $-\infty$ above
- C5.** In masked attention, the mask is:
 (a) multiplied with the attention weights after the softmax (b) added to the scaled scores before the softmax (c) added to the input embeddings (d) applied to the values V
- C6.** In cross-attention, the queries are computed from:
 (a) the decoder's masked self-attention output (b) the encoder output (c) the positional encodings (d) the vocabulary matrix W_D
- C7.** With $T_t = 7$ target and $T_s = 10$ source tokens, one cross-attention head's weight matrix is:
 (a) 10×10 (b) 7×7 (c) 7×10 (d) 10×7
- C8.** Which statement about the decoder's cross-attention is *true*?
 (a) it uses the causal mask (b) it uses a different E for each layer (c) its keys come from the decoder (d) it needs no causal mask because the source is fully known
- C9.** The Euclidean distance between the one-hot position vectors for positions 3 and 50 is:
 (a) $\sqrt{2}$ (b) 47 (c) 1 (d) 2
- C10.** The positional encoding p_0 equals:
 (a) all zeros (b) $[0, 1, 0, 1, \dots]$ (c) all ones (d) $[1, 0, 1, 0, \dots]$
- C11.** With $d_{\text{model}} = 512$, how many distinct frequencies does sinusoidal PE use?
 (a) 512 (b) 128 (c) 256 (d) 1024
- C12.** The longest wavelength in the base model's PE is approximately:
 (a) 2π (b) 512 (c) 10,000 (d) $2\pi \times 10^4$ (about 60,000)
- C13.** For sinusoidal PE, $\|p_j - p_k\|$ depends on:
 (a) j and k separately (b) only $|j - k|$ (c) only $\max(j, k)$ (d) the word embeddings
- C14.** Layer normalization computes its mean and variance over:
 (a) the neurons of one layer, for one sample (b) the samples of the batch, for one neuron (c) the time steps (d) all layers of the network
- C15.** Batch norm with batch size $m = 1$ outputs (per neuron):
 (a) NaN (b) the input unchanged (c) β , because $\hat{x}_i = 0$ (d) γ
- C16.** By the slides' counting, how many layers deep is the base transformer?
 (a) 6 (b) 12 (c) 24 (d) 42

- C17.** The main purpose of residual connections is to:
(a) reduce parameters (b) ensure gradient flow through the deep network (c) encode position (d) normalise activations
- C18.** W_D for $d_{\text{model}} = 512$, $|V| = 37,000$ has about:
(a) 19M parameters (b) 2M (c) 37M (d) 65M
- C19.** The FFN in one layer ($512 \rightarrow 2048 \rightarrow 512$, with biases) has about:
(a) 0.5M (b) 1M (c) 2M (d) 4M
- C20.** The output of the top encoder layer is given to:
(a) only the first decoder layer (b) only the last decoder layer (c) the final linear layer
(d) every decoder layer's cross-attention
- C21.** In the original transformer, the input word embeddings are:
(a) pretrained Word2Vec vectors (b) learned from scratch during training (c) one-hot vectors (d) sinusoidal functions
- C22.** How many Add & Norm blocks are in one decoder layer?
(a) 1 (b) 2 (c) 3 (d) 4
- C23.** Scores are divided by $\sqrt{d_k}$ because:
(a) $\text{Var}(q \cdot k) = d_k$, and large scores saturate the softmax (b) it reduces the parameter count
(c) it implements the mask (d) it normalises over the batch
- C24.** To generate a 20-token translation at inference, the model runs:
(a) 20 encoder and 20 decoder passes (b) 1 encoder and 1 decoder pass (c) 20 encoder passes and 1 decoder pass (d) 1 encoder pass and 20 decoder passes

Part C: Answer Key with Explanations

- C1.** (b) One probability per vocabulary word at each position. (c) is the shape of W_D , not of the output.
- C2.** (c) Masked self-attention, cross-attention, FFN. (d) would count an attention sublayer twice.
- C3.** (a) The true *previous* token. (d) would hand the model the answer; (b) is free-running mode.
- C4.** (d) Zero where attention is allowed (past and present), $-\infty$ for future positions.
- C5.** (b) Adding $-\infty$ before the softmax gives exactly 0 weight *and* keeps each row summing to 1.
- C6.** (a) $Q_2 = W_{Q_2}S$, where S is the output of the masked self-attention sublayer.
- C7.** (c) Rows correspond to queries (target), columns to keys (source).
- C8.** (d) The source sentence is entirely available, so there is nothing to hide. The same top-layer E goes to every layer.
- C9.** (a) Any two distinct one-hot vectors differ in exactly two entries: $\sqrt{1+1}$.
- C10.** (b) $\sin 0 = 0$ in even entries and $\cos 0 = 1$ in odd entries.
- C11.** (c) Each frequency index $i = 0..255$ fills a $(\sin, \cos)$ pair.
- C12.** (d) $\lambda_{255} = 2\pi \cdot 10000^{510/512} \approx 60,600$. (c) forgets the 2π .
- C13.** (b) $\|p_j - p_k\|^2 = \sum_i [2 - 2\cos(\omega_i(j - k))]$.
- C14.** (a) This is the defining difference from batch norm, which is (b).
- C15.** (c) $\mu_i = x_i$ makes $\hat{x}_i = 0$ (ϵ prevents division by zero, so not NaN); then $\gamma \cdot 0 + \beta = \beta$.
- C16.** (d) $6 \times (1 + 2) + 6 \times (2 + 2) = 42$.
- C17.** (b) The identity path gives $\partial x_{k+1} / \partial x_k = I + \partial F / \partial x$.
- C18.** (a) $512 \times 37,000 = 18,944,000$.
- C19.** (c) $2(512 \cdot 2048) + 2048 + 512 = 2,099,712$.
- C20.** (d) Stated on the final slide: the top encoder output feeds all decoder layers.
- C21.** (b) No pretrained embeddings were used. They are an additional set of learned weights.
- C22.** (c) One after each of the three sublayers.
- C23.** (a) Dividing by $\sqrt{d_k}$ brings the score variance back to 1.
- C24.** (d) The encoder runs once. The decoder runs once per generated token (autoregressive).

Answer distribution: (a) $\times 6$, (b) $\times 6$, (c) $\times 6$, (d) $\times 6$.